

#	Section
1	Objective and problem statement
2	Class diagram (hand-drawn UML)
3	Worked trace: fixed vs. sliding window
4	Sample output / report
5	OOP concept mapping and conclusion

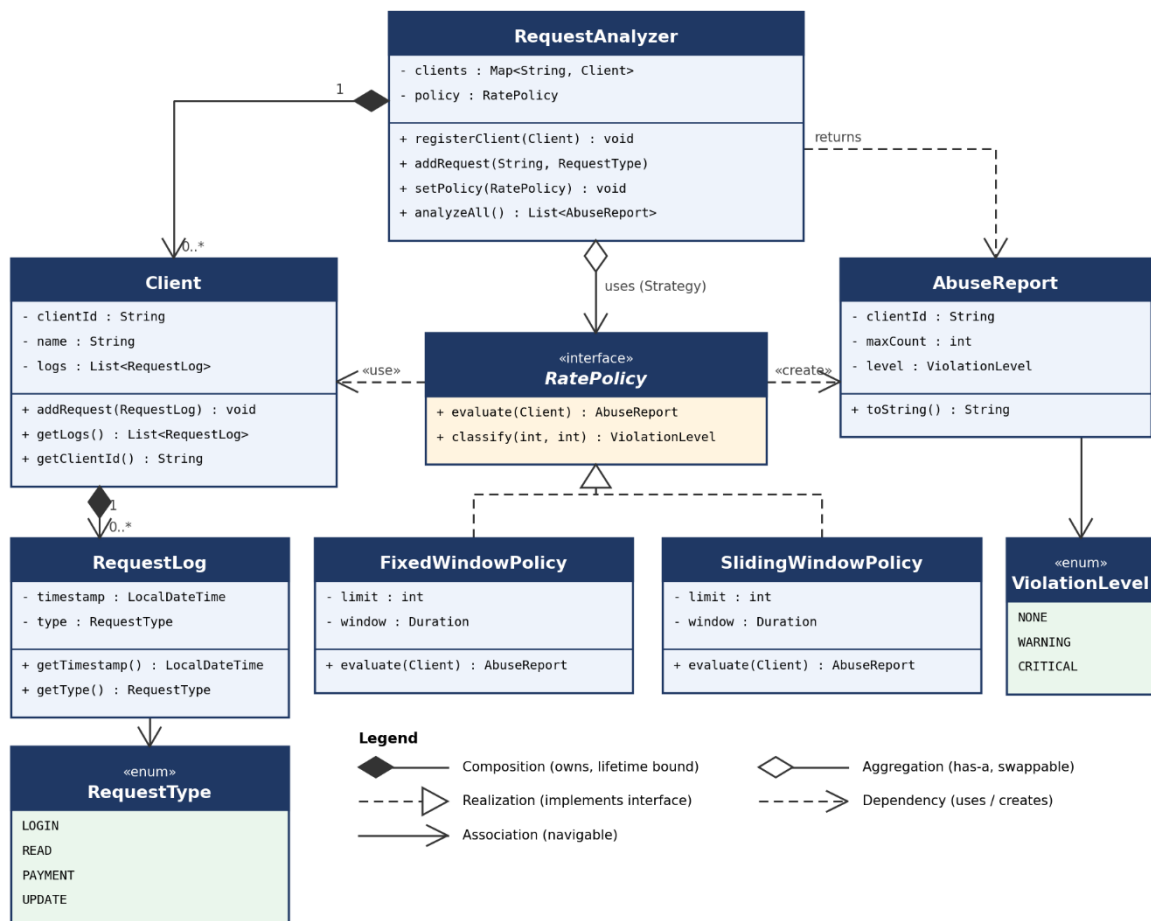
1. Objective

Write 3–4 lines, for example:

To build a JavaFX simulation of how a service tracks client requests in memory, applies rate-limit policies (fixed and sliding window), detects abusive usage, and produces a violation report for each client. There is no networking; it is a request log plus a rule engine.

2. Class Diagram (Hand-Drawn UML)

The diagram must show **composition** (a Client has a list of RequestLogs) and **realization** (both policies implement the RatePolicy interface).



3. Worked Trace: Fixed vs. Sliding Window

Policy: 5 requests per 10 seconds. CLIENT_A sends requests at:

10:15:36, :37, :38, :39, :39 | :40, :40, :41, :42, :43

Policy	Window(s) examined	Max count	Result
Fixed	[:30–:40) = 5, [:40–:50) = 5	5	NONE (passes)
Sliding	(:33, :43] = 10	10	10/5 = 2.0 → CRITICAL

The fixed window has a **boundary-burst weakness**. A client can send 2× the limit across a window edge. The sliding window closes this gap, but it costs more: $O(n)$ memory to keep timestamps, compared with $O(1)$ counters per window. This is the trade-off real API gateways weigh, and it is why production systems use approximations such as the sliding-window counter or the token bucket.

4. Sample Report Output

Client	Policy	MaxCount	Limit	Level
CLIENT_A	Fixed	5	5	NONE
CLIENT_A	Sliding	10	5	CRITICAL
CLIENT_B	Sliding	3	5	NONE

5. OOP Concept Mapping and Conclusion

Concept	Where it appears
Encapsulation	limit and window are private; logs can only be reached through Client
Polymorphism	RatePolicy p = new SlidingWindowPolicy(...)
Strategy pattern	Policies can be swapped at runtime without changing the analyzer
Enums	Fixed request types and severity levels make invalid states impossible
Java Time API	Duration.between() does window arithmetic without manual seconds math

Conclusion (2–3 lines): the simulation shows that abuse detection comes down to counting events in time windows, and that the choice of window algorithm affects both accuracy and cost.